

Print it yourself.

The customisation system, end to end — the DEWDROPZ design library, the studio it opens in, the way a printed garment points back at the studio it came from, and every upgrade behind all three.

105 COMMITS 4 MIGRATIONS 3 SURFACES 300 DPI TARGET 0 OFF-LADDER RADII

CONTENTS

- | | |
|--|---|
| 01 The two doors | 02 The design library |
| 03 Managing it from the back end | 04 The custom range |
| 05 Blanks and print zones | 06 The studio |
| 07 The print pipeline | 08 The phone |
| 09 Data model | 10 UI and design system |
| 11 What shipped, when | 12 Runbook |
| 13 Completeness | |

01 The two doors

The brief put it in one sentence:

There will be two options: customer can select from our pre-set design ready library of DEWDROPZ and second — customer can upload their own design.

The second door had existed since the studio shipped. **The first did not exist at all.** Building it is the spine of this phase, and it pulled three other things in with it: a way to *manage* that library, a way to say *which garments* a design belongs on, and a way to mark an already-printed product as part of the custom range so its page can point back at the studio.

Both doors are now named out loud, and the choice survives every hop between the homepage and the canvas — `?start=library` is carried through the index, through each blank card, through the custom-range banner, and into the phone app.

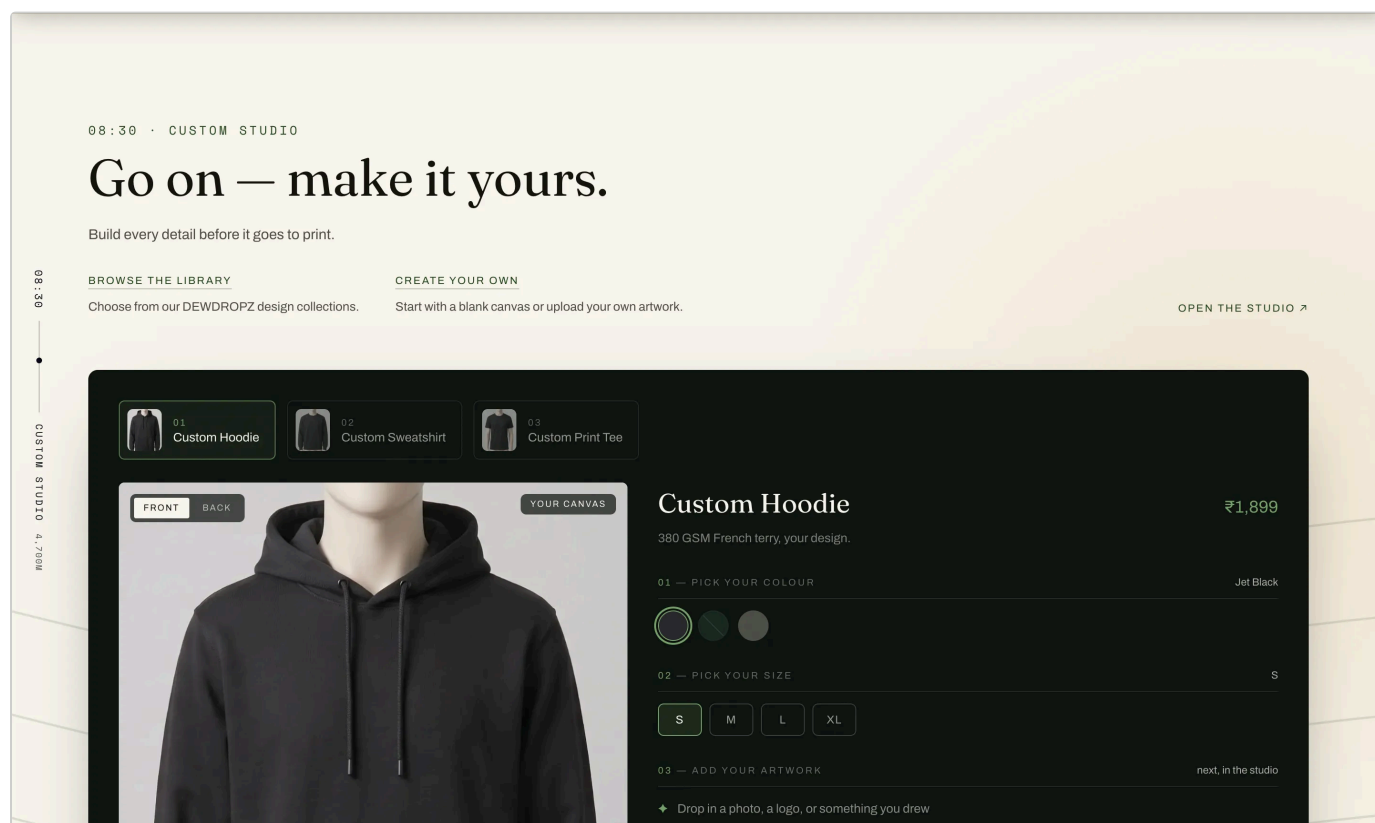


FIG. 01 The homepage section. The two doors are the first thing under the headline; the workbench below is driven off the real catalogue, so a fourth blank flagged customizable in admin appears here on its own.

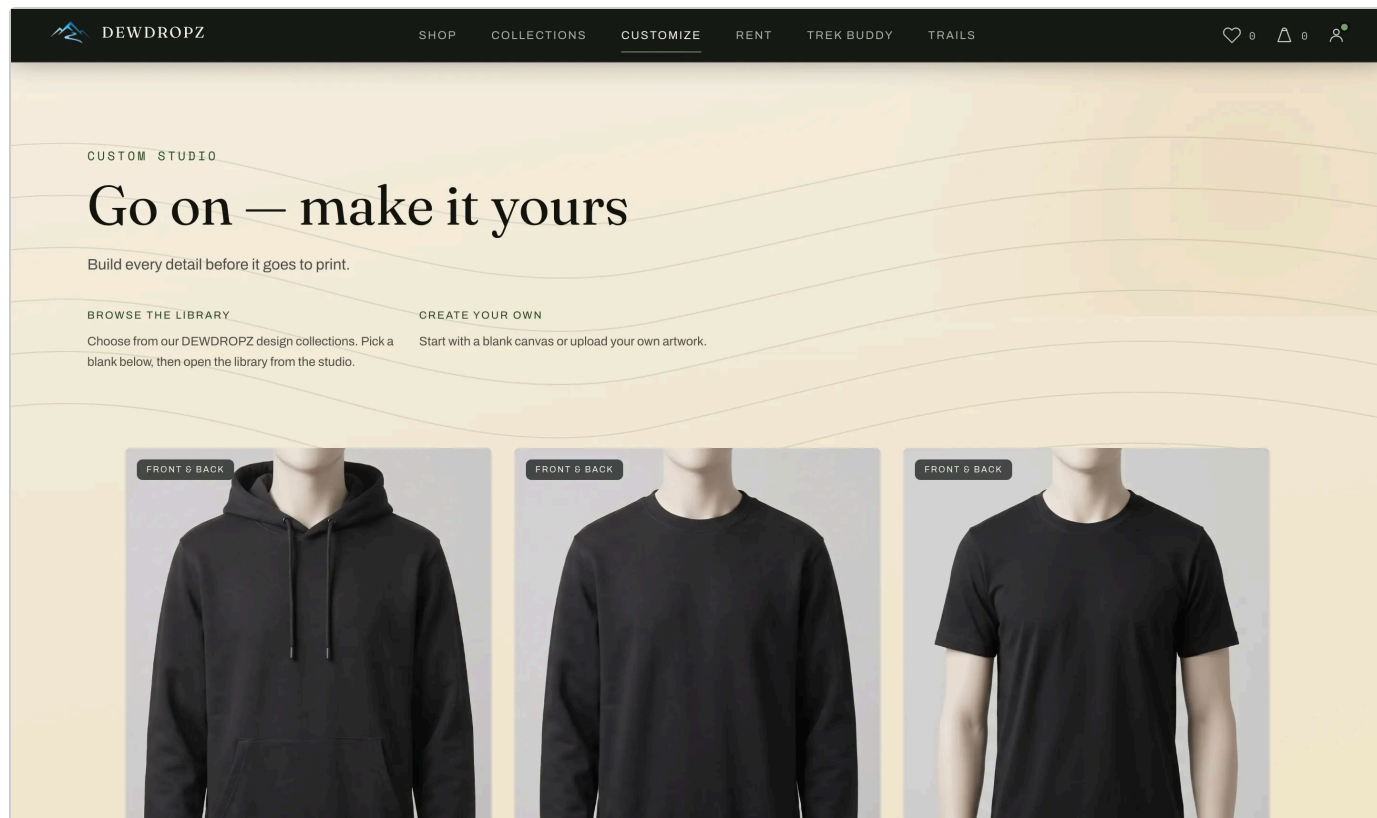


FIG. 02 `/customize`. Previously this page offered one route in — start with a blank canvas — which quietly told everybody who is not a designer that the studio was not for them. Dark cards on a lit ground, because garment photography needs a lit surface and the brand chrome around it is dark.

02 The design library

Artwork DEWDROPZ has drawn, offered in the studio beside the customer's own uploads. It is grouped by design collection, sorted by admin, and filtered to the garment in hand.

It is not a second editor

The critical implementation choice. A library design lands on the canvas through the *exact same* `FabricImage.fromURL` path an upload takes. From the moment it arrives it is movable, scalable, rotatable, deletable, layer-orderable and exported at print resolution — like anything else. The only difference between the two doors is where the URL came from.

Details that took a decision

- **Fetches only when the panel is opened**, never on studio mount. A shopper who brings their own artwork should not pay for a catalogue they will not look at.
- **Checkerboard tiles**. Most library art is transparent PNG, and a transparent design on a dark panel looks like an empty tile.
- **Escape closes it**. The canvas underneath binds Delete and the arrow keys, so a panel sitting over it has to take the keyboard seriously.
- **Grouped by collection, plural**. The brief's "choose from our DEWDROPZ design collections" is the entire reason the column exists.
- **Failure is designed**. A read error resolves to an empty list and the panel says so — the upload door still works, so a library that cannot load must never break the studio.

ONE SCROLL FIX WORTH RECORDING

Lenis is mounted app-wide and swallows wheel events. Without `data-lenis-prevent` the shelf scrolled programmatically but not with a mouse or trackpad — it simply looked stuck. The shadcn primitives all carry it; these hand-rolled studio panels never inherited it.

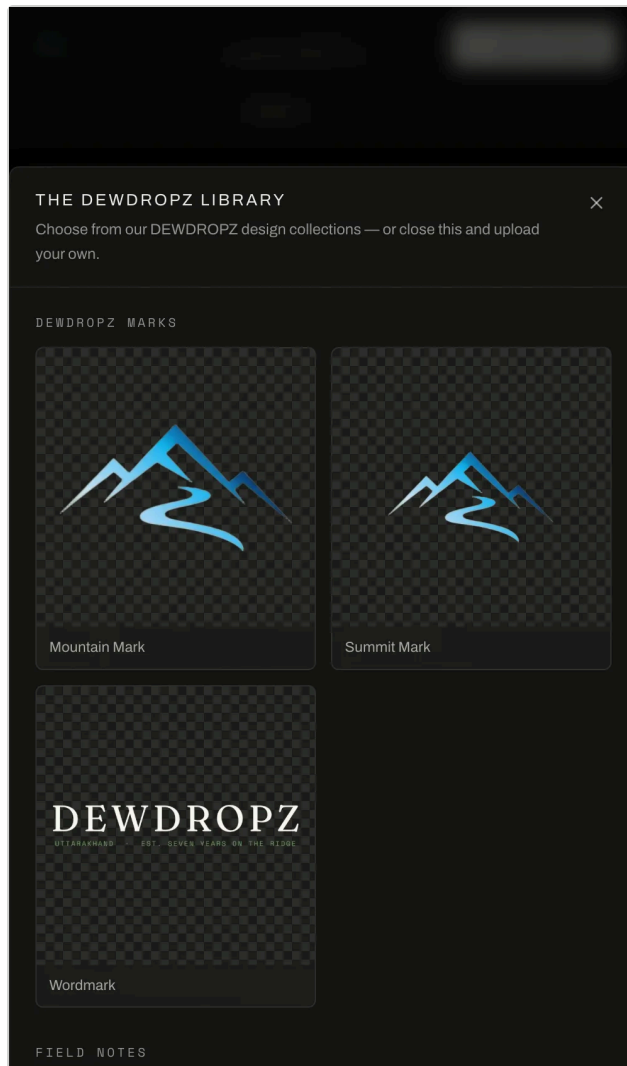


FIG. 03 Bottom sheet on a phone.

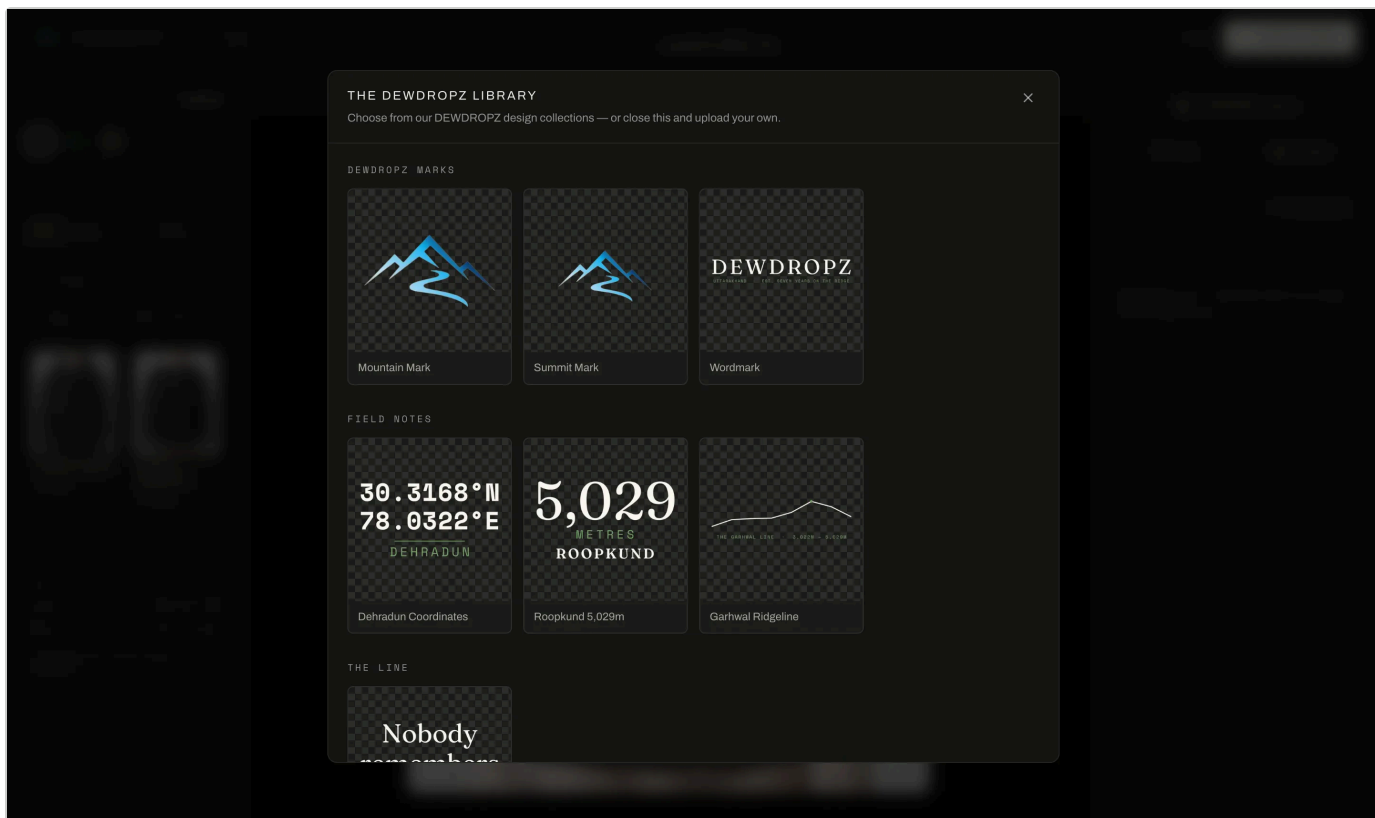


FIG . 04 Centred panel from `sm:` up, grouped into the three collections currently stocked — *DEWDROPZ Marks*, *Field Notes*, *The Line*.

What is in it, and why it is honest

Nothing in the seed is invented artwork. Two entries are the brand marks already in `public/logo/`, uploaded as they are. The rest are set from the brand's own words in the brand's own faces — the same Fraunces and Space Mono TTFs in `assets/fonts/` that the print renderer registers, so a library design and a customer's own text layer come out of the same typefaces on the same press.

TWO CONSTRAINTS RECORDED RATHER THAN HIDDEN

Ink is light on purpose. The only colourway orderable today is Jet Black. Dark artwork on the only buyable garment would be invisible. When a light colourway goes live these need dark counterparts — a real follow-up, not an oversight.

Resolution is honest. Generated marks are authored at 300 DPI across the real 12in zone — 3,600px. The existing logo files are uploaded at native size and *not* upscaled: the mountain mark is 1,425px, about 119 DPI across a full 12in front. Upsampling would manufacture detail that was never there and defeat the warning that exists to catch exactly this. Placed smaller it is sharp, and the studio says so.

03 Managing it from the back end

One screen — `/admin/designs` — owns the whole library. Upload, rename, re-collect, re-sort, switch on and off, restrict to garments, delete.

Per-row saves, no page-wide Save button

The only page-wide thing here is a list, and a list of independent records sharing one Save is how you lose four edits to one failed upload. Each row patches optimistically and rolls back on failure.

The screenshot shows the DEWDROPZ Admin interface for the Design Library. The left sidebar contains navigation links: OVERVIEW (Dashboard, Analytics), CATALOG (Products, Collections, Categories, Tags, Attributes, Design Library, Rentals, Homepage), and SALES (Orders, Payments, Coupons, Promotions, Print Queue, Returns, View Storefront). The main content area is titled 'Design Library' and includes a sub-header 'Add a design' with instructions: 'PNG with a transparent background is what the studio wants — it goes onto black, sage and sand garments alike. Up to 10MB. Aim for at least 2000px on the long edge so it still prints at 300 DPI across a chest.' Below this are input fields for 'Name' (containing 'Ridge line') and 'Design collection' (containing 'DEWDROPZ'). There is an 'Artwork' section with a 'Choose file' button and a 'No file chosen' status. An 'Add to library' button is at the bottom. The 'In the library' section shows a list of designs, with the first one being 'Mountain Mark' by 'DEWDROPZ MARKS', sorted by '10', and marked as 'Live'. It also shows 'Offered on' options: 'Every garment', 'Custom Hoodie', 'Custom Print Tee', and 'Custom Sweatshirt'.

FIG. 05 The upload card states the rule and the reason: PNG with a transparent background, because it goes onto black, sage and sand garments alike; up to 10 MB; at least 2000px on the long edge so it still prints at 300 DPI across a chest.

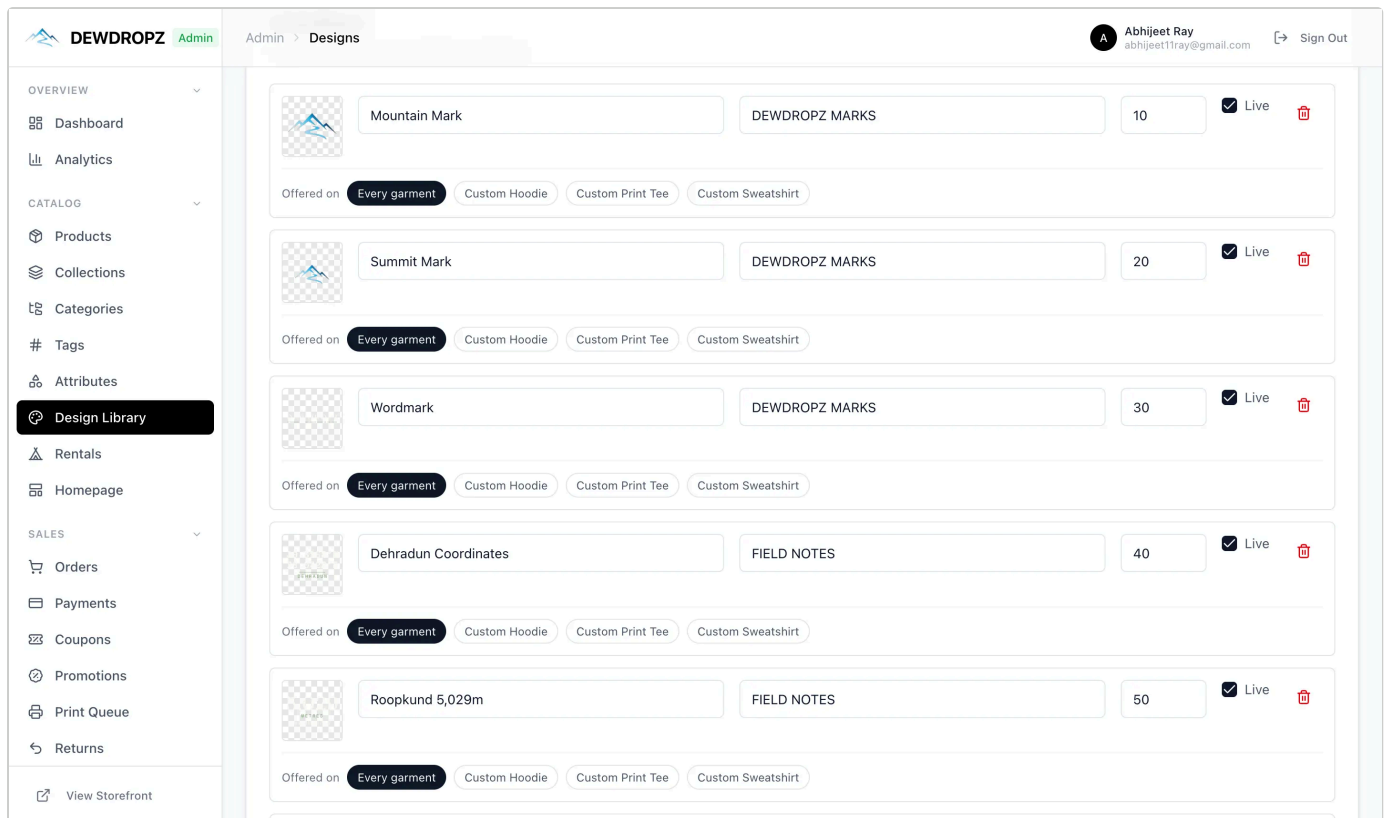


FIG . 06 Scoping. **Empty means every garment**, and the control says so rather than rendering as “none selected” — that ambiguity is the whole reason to spell it out instead of leaving a bare multi-select. The server de-duplicates the array and verifies every id is a real, undeleted, customizable blank before writing.

Orphan discipline, in both directions

If the row insert fails after a successful upload, the object is deleted — that is how a storage bucket quietly fills with files nothing points at. On delete, the *row* goes first and the object second, best-effort: a stranded object costs storage, whereas a row pointing at a deleted object is a broken image in the studio. If only one of the two can happen, that is the right order for it.

Slugs, without asking the admin to think

Derived from the name, lowercased, non-alphanumerics collapsed to hyphens, truncated to 48 characters, plus a six-character random tail — which satisfies both the shape constraint and the uniqueness constraint on its own.

04 The custom range

The studio and the catalogue had never known about each other. A **blank** opens the studio; a finished, already-printed garment is an ordinary product row with its own photographs. Nothing joined the two, so a printed tee could not say where it came from, and a shopper who liked the shirt but not the artwork had no route to “what else can go on this”.

The tick is the switch; the parent is optional

The first attempt modelled a printed garment as a *recipe* — a blank plus a library design. That is not how these products are made. They are photographed, not composed: an admin uploads a picture of a shirt that has already been printed, and the only thing they want to say about it is that it belongs to the custom range. Requiring them to also pick the artwork out of a library — artwork that may not *be* in the library, because the print was a one-off — makes them describe the product twice, in a vocabulary the product does not have.

So `is_custom_range` is a plain boolean an admin ticks, and it alone decides whether the storefront offers the studio. `custom_blank_id` stays as an optional parent link.

WHY A SEPARATE FLAG, AND NOT "CUSTOM_BLANK_ID IS NOT NULL"

Because the two facts are genuinely different, and the difference is the whole feature. “This is a custom-range product” is a merchandising decision. “It was printed on that blank” is a fact that may be unknown, or may become false when a blank is retired — which is exactly when you most want the page to keep saying *this came out of our studio*.

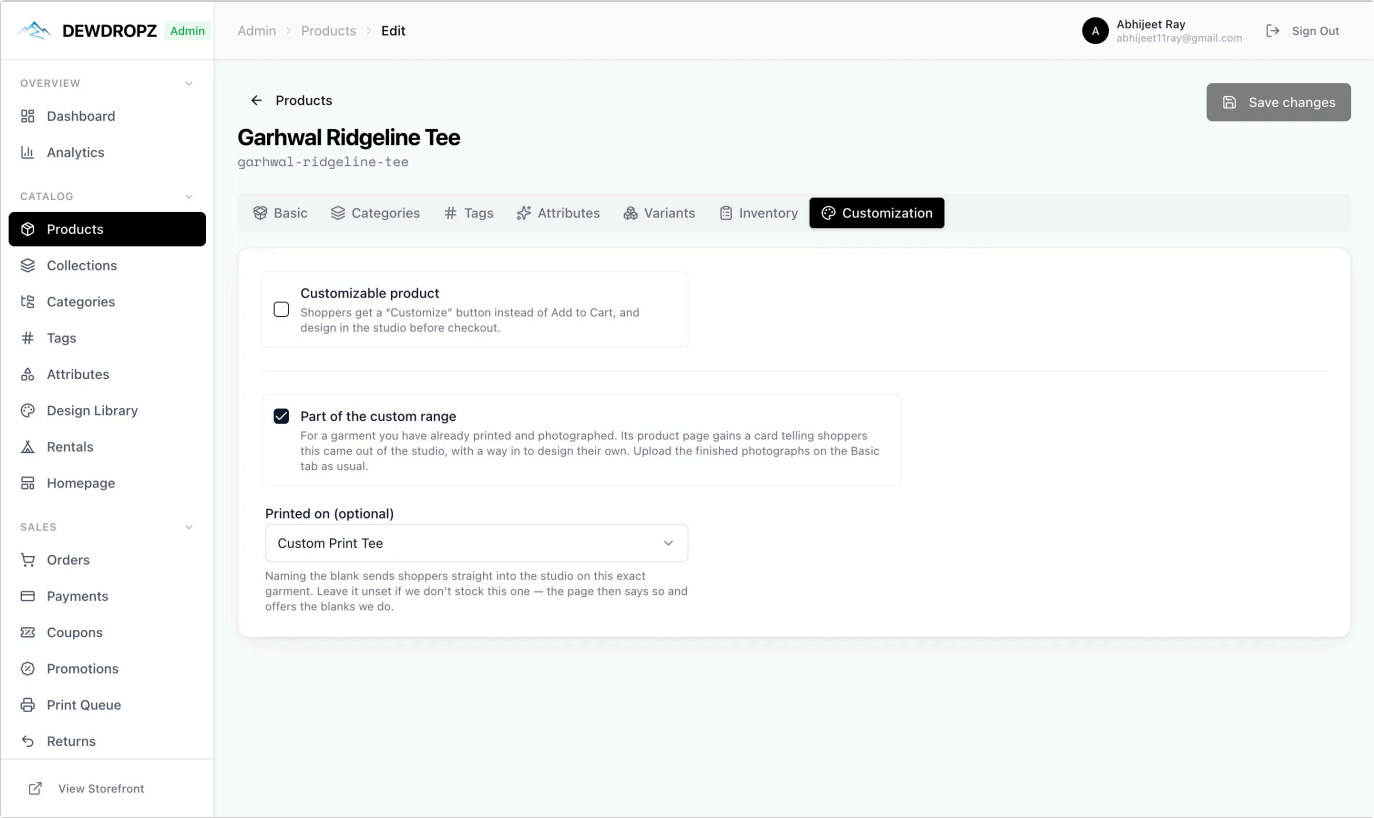


FIG . 07 Marking a product. The two halves are mutually exclusive and the UI enforces it before the database has to: a blank is the thing designs go *on*, so it cannot also be a print *of* one. Unticking the range clears the parent, so an untick never produces a save that bounces.

Two honest states, one card

The storefront card changes its wording rather than its presence. The stale-link case is the one worth getting right: the parent is re-checked for `is_customizable` and `is_active` on every render, and if either is false it is treated exactly like “not stocked”. A stale foreign key must not become a broken promise on a product page.

STATE	WHAT THE SHOPPER GETS
Parent blank stocked and live	“Printed on the <i>Custom Print Tee</i> — put your own artwork on the same garment.” Straight into the studio, or into the studio with the library already open. Plus a rail of other prints on the same garment.
No parent, archived, or customization switched off	“Want this with your own artwork on it?” — says so plainly and offers the blanks that <i>do</i> exist.

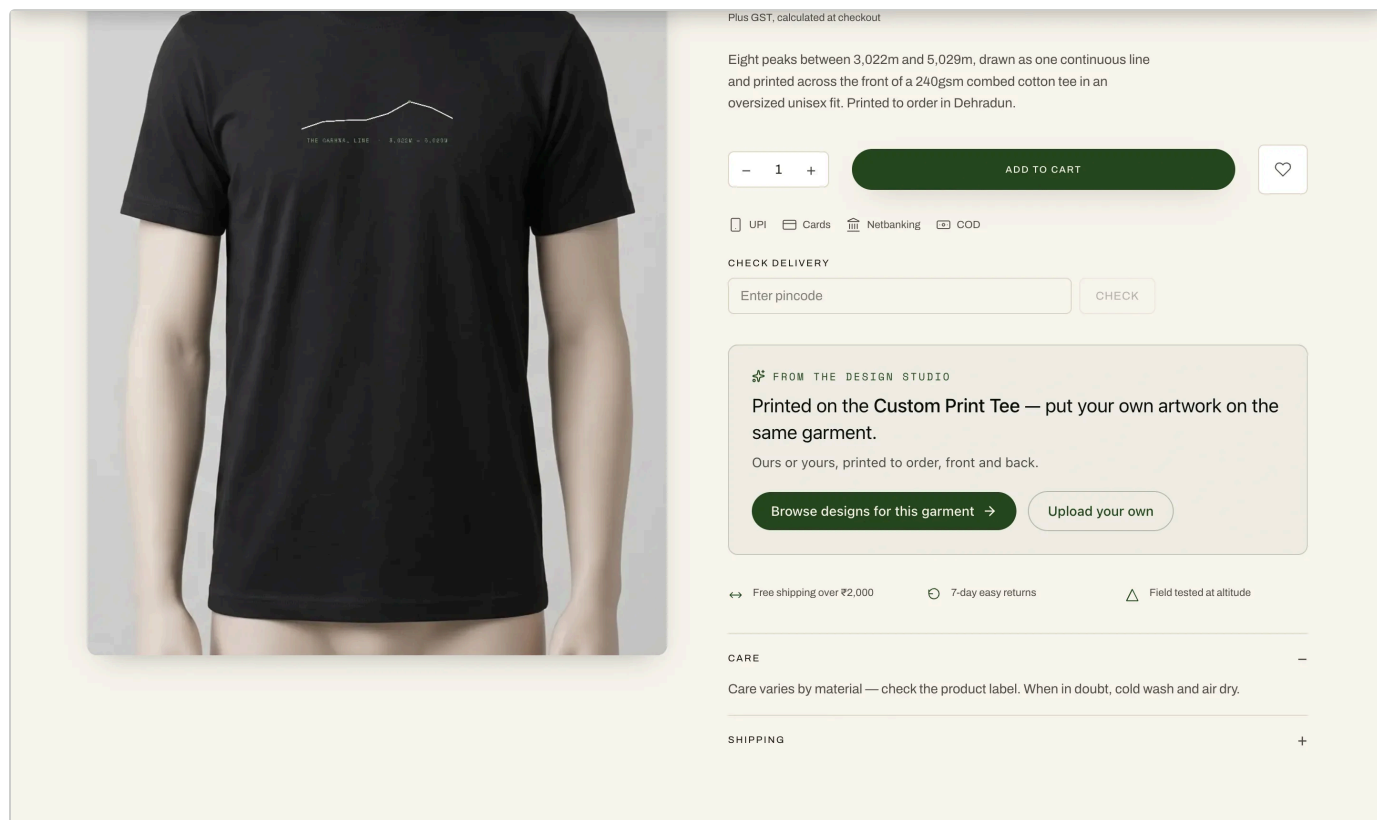
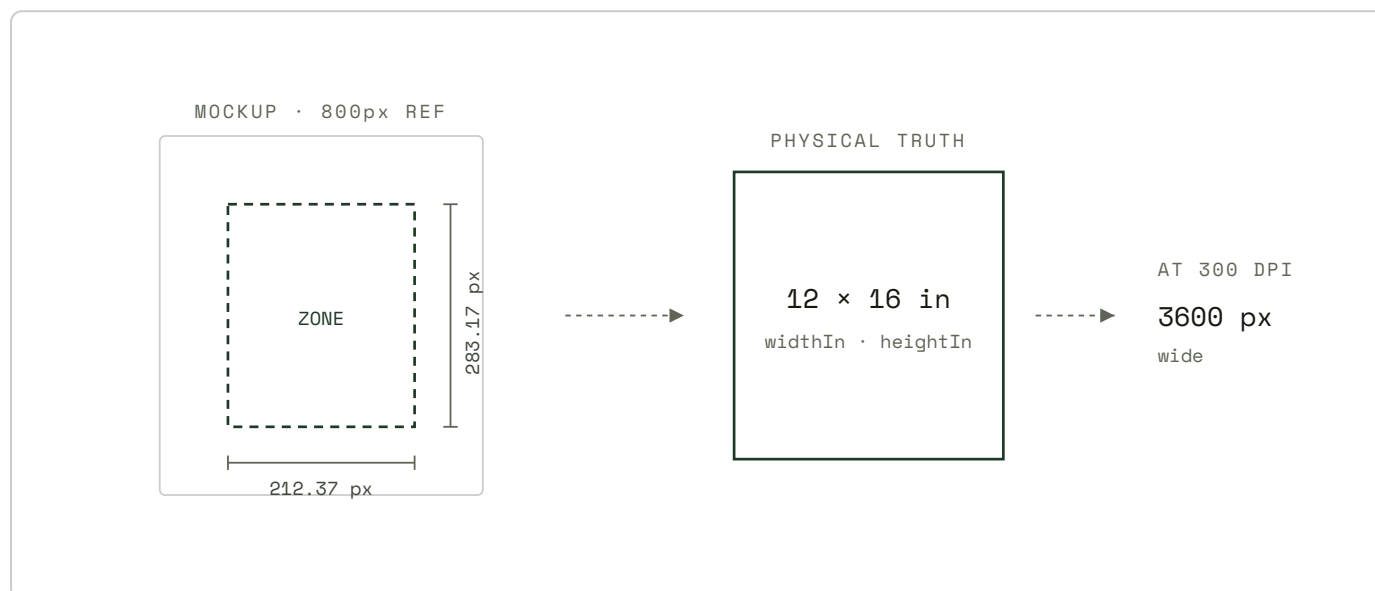


FIG . 08 The card on a range product. The product photograph itself is a real composite — the actual Jet Black mockup with the actual library artwork drawn into the actual print zone read from the garment's config, in the same canonical 800px space the studio and the renderer use. It shows what the press would produce, not a picture of a shirt that does not exist.

05 Blanks and print zones

A blank is a product with `is_customizable` and at least one colourway carrying print zones. A blank with no zones never appears anywhere — every surface filters on it, because a studio with nothing in it is worse than no link at all.

The zone is the object every part of the system agrees on, and it records **two coordinate systems at once**:



Canvas pixels are arbitrary and tied to the 800px reference mockup. Inches are physical truth. Every resolution decision in the system is computed from the physical size alone — 12 inches at 300 DPI is 3,600 pixels, whatever the canvas happens to be.

Zones genuinely differ per garment. The tee's front is 212.37 × 283.17px standing for 12 × 16in; the hoodie's is 219.83 × 292.48px for the *same* 12 × 16in. That is why carrying a design between blanks needs real geometry rather than a copy-paste, and why moving one onto a bigger garment can quietly drop it below the resolution floor.

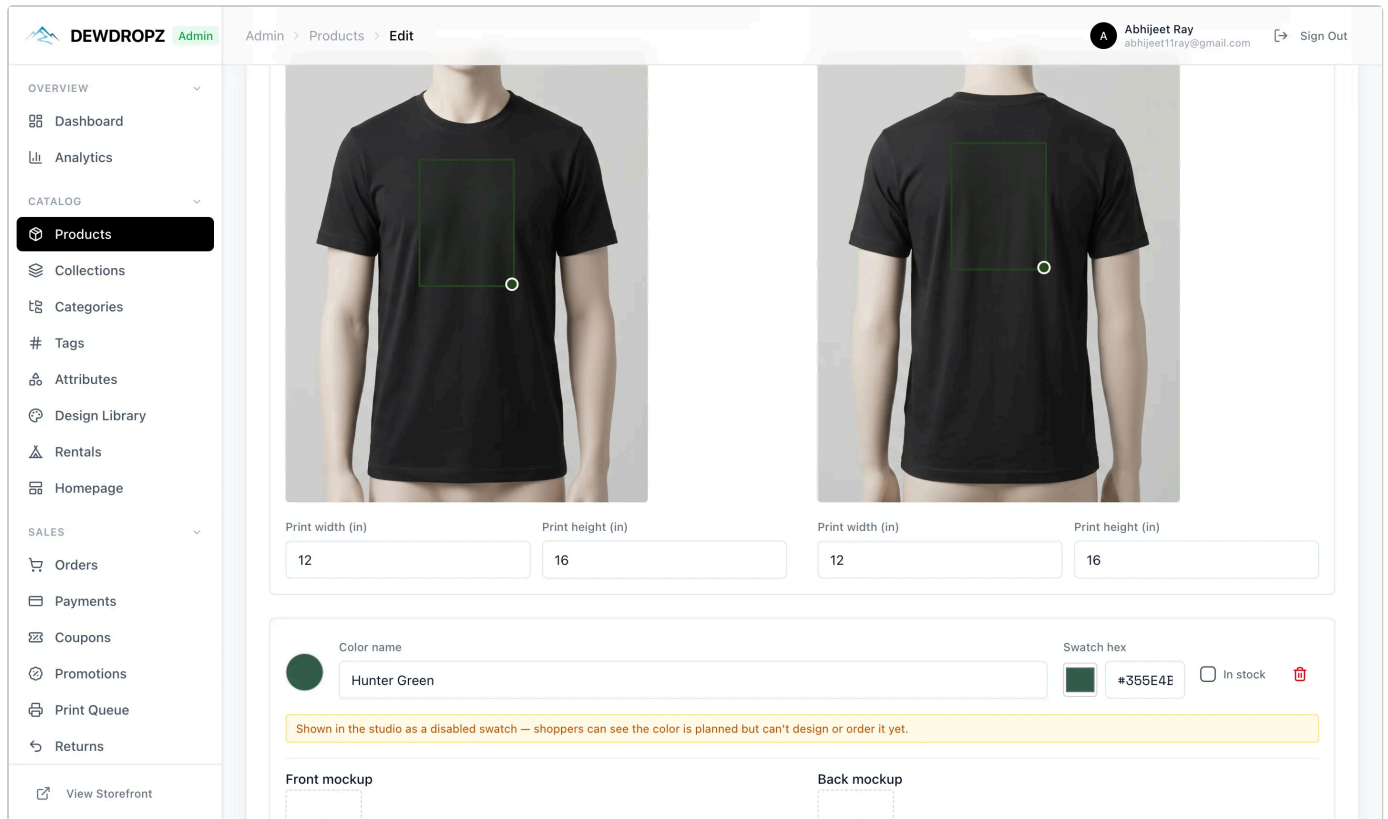


FIG . 09 Where zones are authored. Drag the box to position the print area, drag the corner to resize — and this is exactly what shoppers see in the studio. The *Print width / height (in)* fields are the physical half of the pair above; everything downstream is derived from them. Below, a colourway marked out of stock, which the studio renders as a disabled swatch so a shopper can see a colour is planned without being able to order it.

06 The studio

Laid out like the design tools people already know: a setup rail on the left — *what garment am I making* — the stage in the middle, and tools on the right — *what am I putting on it*. Dark chrome throughout, so the garment and the artwork are the only bright things on screen.

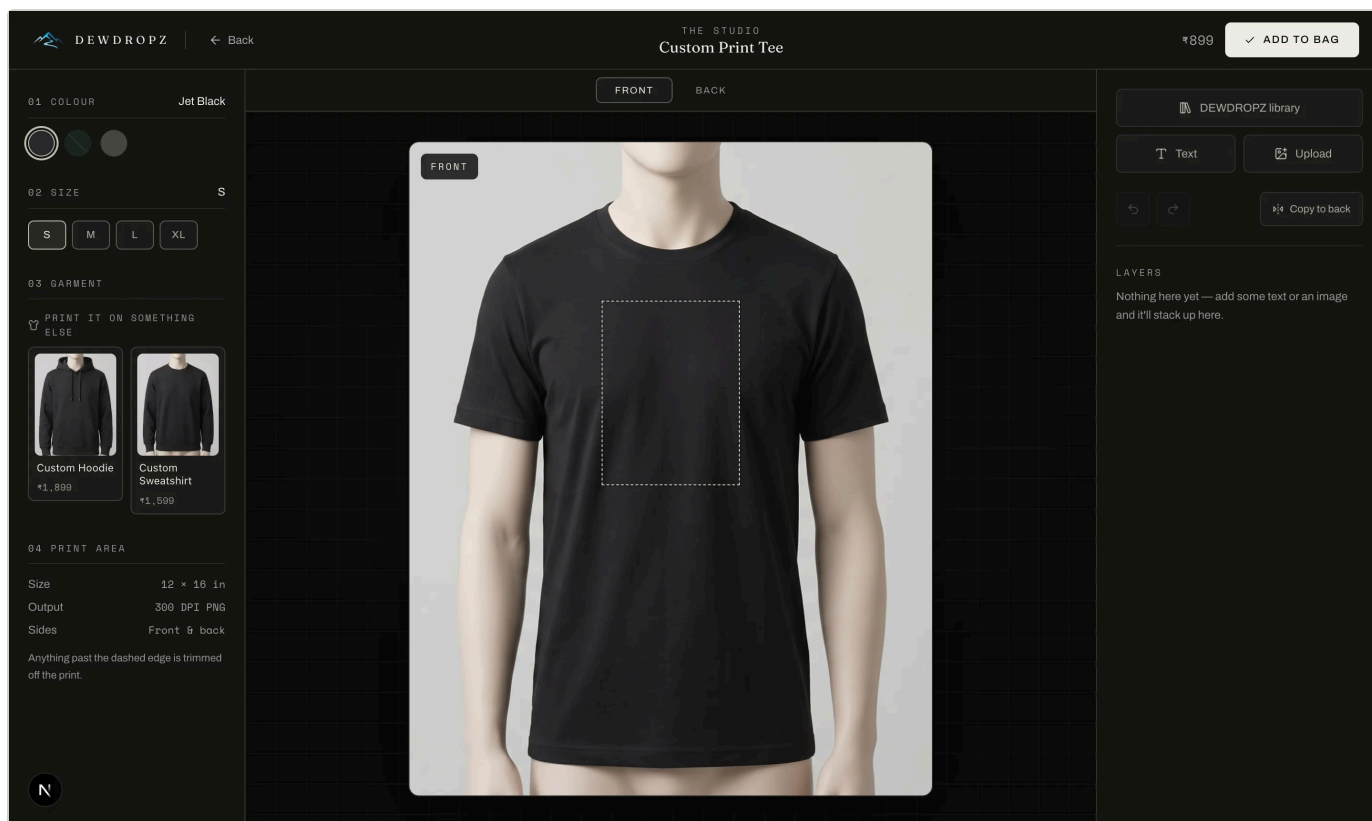


FIG. 10 Empty state. The left rail numbers the four setup decisions and ends with the print spec in plain terms — *12 × 16 in · 300 DPI PNG · Front & back*, and “anything past the dashed edge is trimmed off the print”. The layers panel says what will happen rather than sitting blank.

Front and back are a tab pair

Both canvases are mounted; one is shown. The side you are *not* looking at carries a filled dot when there is work on it — without that cue, a design on the back is invisible from the front. **Copy to back** duplicates the active side in one action.

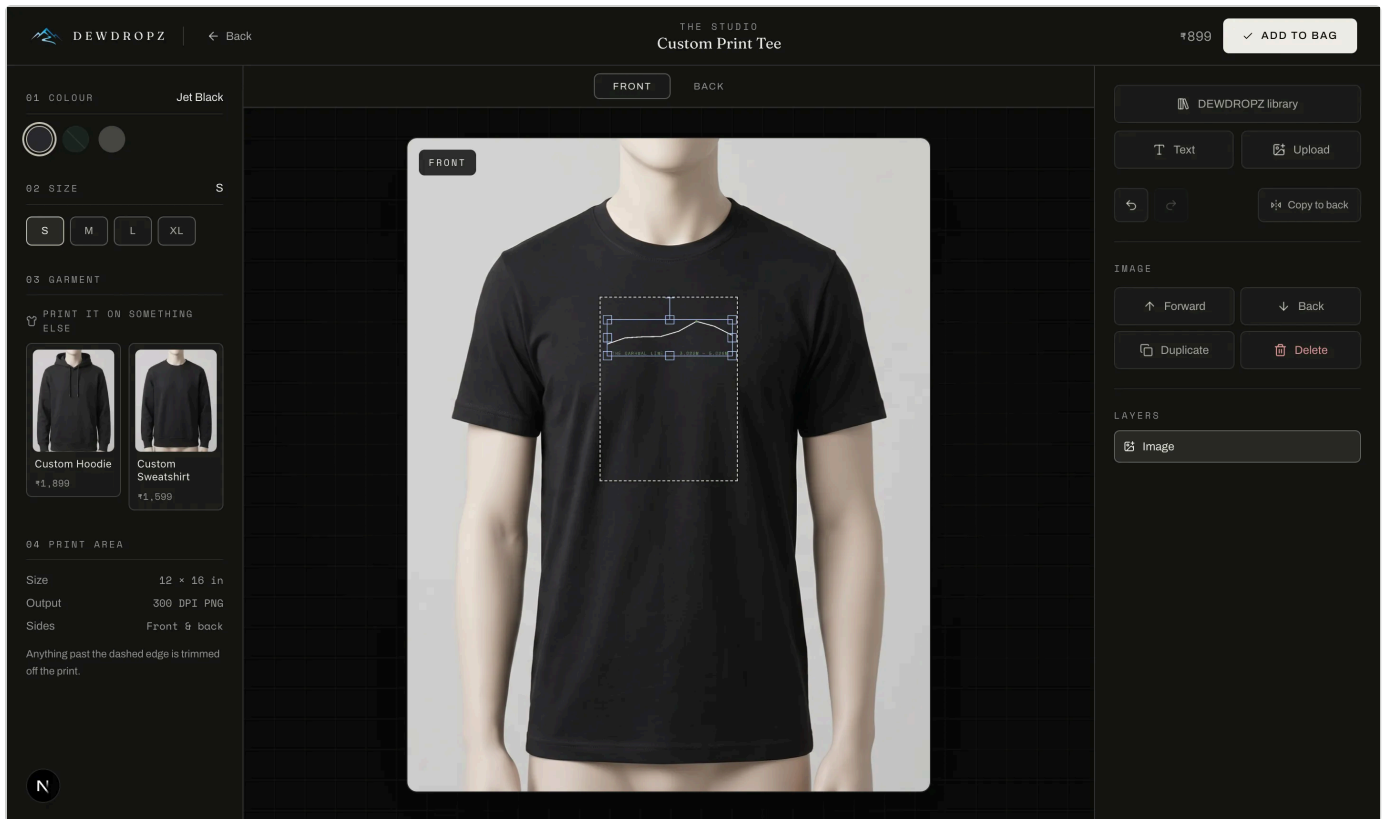


FIG . 11 A library design, placed. Identical to an upload from this moment on.

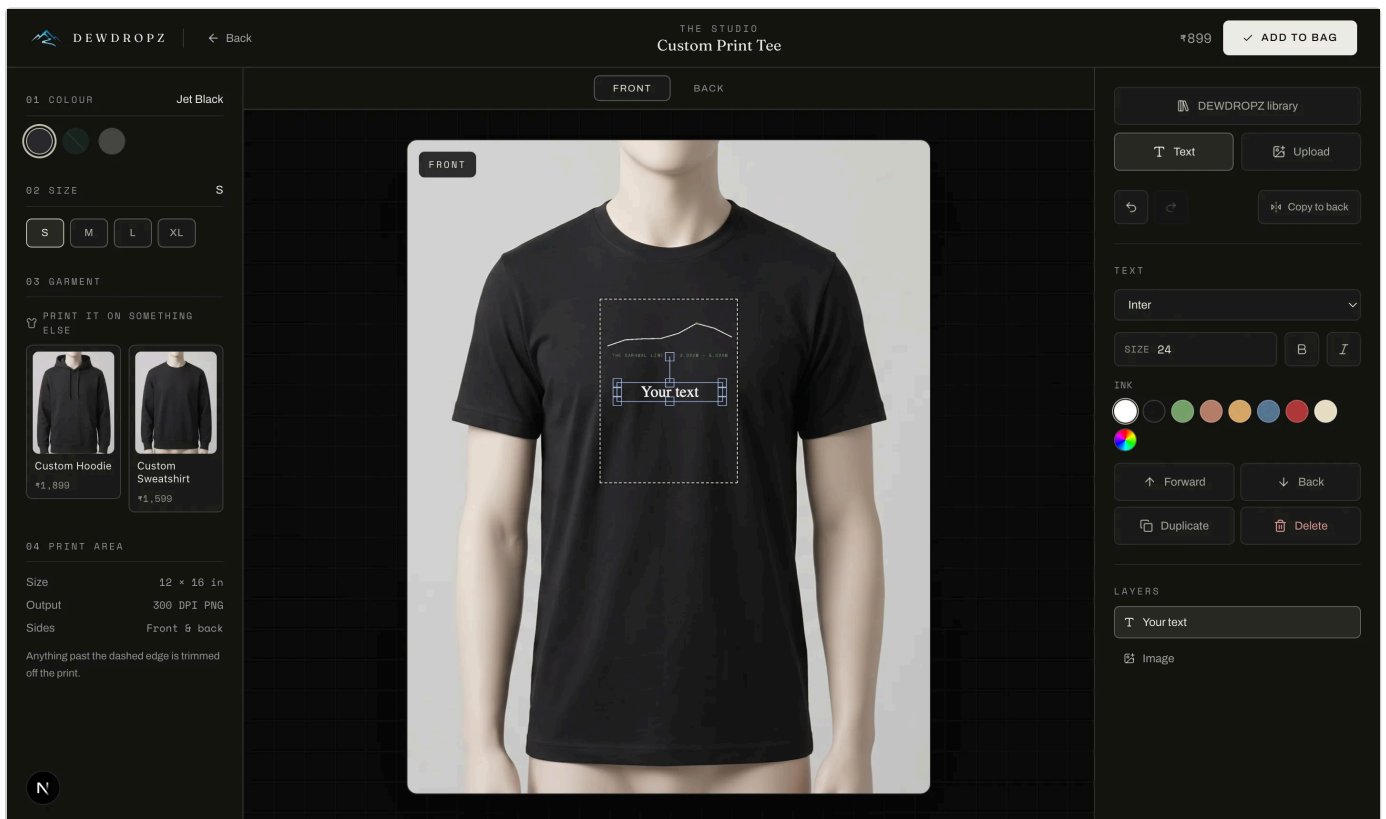


FIG . 12 A text layer. Note the ink: it defaulted to *white*.

WHY THE INK DEFAULTED TO WHITE

New text is coloured by the garment's luminance — Rec. 709 relative luminance on the colourway hex decides black ink or white. Without it, text defaulted to near-black and landed invisibly on the black garment, which is the default colourway, so it hit *every* shopper.

Two more that were caught by measuring, not looking

- **New objects are positioned against the zone's canonical size, never `canvas.getWidth()`.** That reports CSS width — 94px on a phone, 162px on a laptop. Centring off it put new text at a negative x on small screens, so the first characters fell outside the print area and were clipped out of the exported file.
- **Styling changes are pushed into undo history explicitly.** Fabric's `.set()` is silent, so without a manual fire a font, size or colour change was invisible to undo — you could never take a styling change back. Stacking order goes in too; it is part of the design.

What the shopper can do

CAPABILITY	DETAIL
Two doors	<code>?start=library</code> opens the DEWDROPZ shelf; anything else opens a blank canvas
Own artwork	Upload, straight onto the canvas
Library artwork	Same code path, indistinguishable thereafter
Text	Five faces, size, bold, italic
Ink	A curated set that looks good on a garment, plus a full picker
Transform	Move, scale, rotate, flip horizontal
Layers	Live list, click to select, reorder, duplicate, delete
History	Full undo and redo
Sides	Front and back independently, with copy across
Garment	Changed from inside the studio, carrying the work with it
Colourway & size	Filtered to what is actually orderable; carried into the cart line

Carrying a design between garments

Until this shipped, the only route from the tee to the hoodie was: leave the studio, go to the shop, find the other blank, open it — four navigations, and the work in progress was gone by the second. So people did not explore garments; they picked one at the door and lived with it. The left rail now carries *Print it on something*

else right beside the colour swatches — the two things that change what you are printing on, together. It renders nothing when there is only one blank, because a picker with one option is furniture.

THE TRANSPORT IS EASY; THE GEOMETRY IS THE POINT

A canvas with one photograph on it serialises to hundreds of kilobytes — a URL cannot hold that and neither can a cookie. It rides in `sessionStorage` with a 30-minute expiry: per-tab, never sent to the server, long enough to switch garment and short enough that a tab left open overnight does not resurrect yesterday's artwork.

Layer coordinates live in the *source zone's* pixel space, so the carry records the zone it came from and re-fits everything proportionally on arrival.

And the part that costs money: print resolution is a function of the zone, not the image. The same photograph that prints at 300 DPI across a 10in placement drops below the floor when a bigger garment lets it spread wider. The arriving studio re-runs the check and, if the artwork grew, says so: *"Brought over from the Custom Print Tee and resized to fit. This garment prints larger, so check the quality warning before adding to cart."* Silently carrying a design onto a bigger garment is exactly how a shopper ends up approving a blurry print.

On a phone

The left rail folds into one shared bottom sheet with four tabs, of which exactly one is open at a time — and *none* is a legitimate resting state. That is the whole point of the mobile pass: the stage is the default, tools are transient. Tab targets are 56px, comfortably past the 44px minimum, because that bar is the studio's primary navigation and sits in the thumb zone. Open a panel and the stage box gets shorter, so the garment scales down and stays wholly visible.

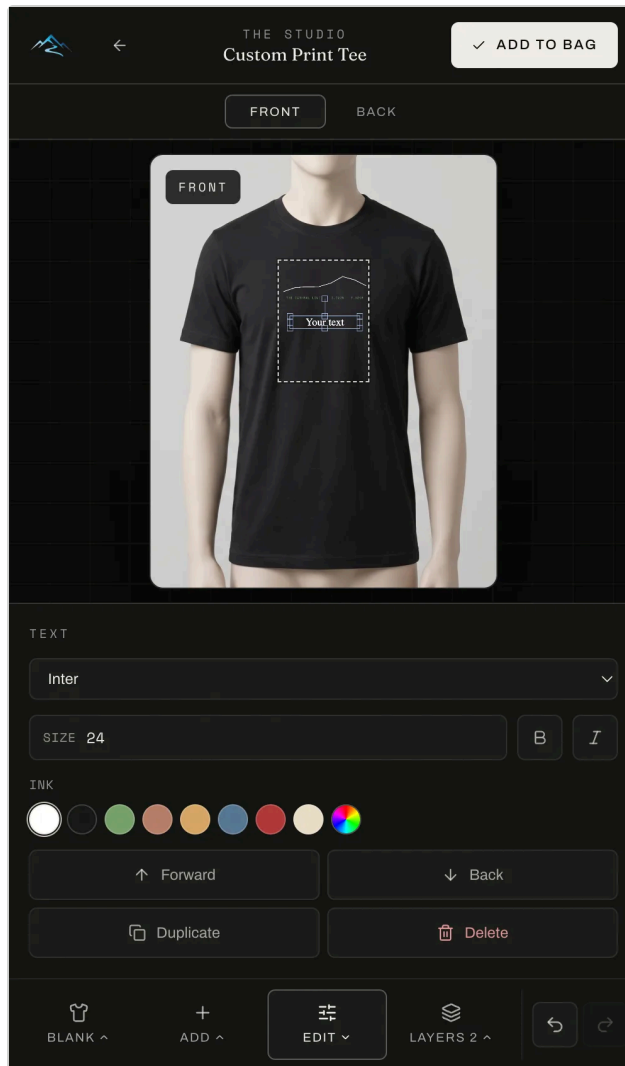


FIG . 13 390 × 844. The tab bar counts the layers so you know there is work behind a closed panel.

07 The print pipeline

There is now **one definition of what a print file has to be**, and it exists because there were two renderers and they disagreed.

THE BUG THAT FORCED THE UNIFICATION

The web studio derived its export scale from the zone's physical size and hit 300 DPI. The server renderer behind the mobile design API used a hardcoded `PRINT_SCALE = 4`, which on the tee's 212px zone produced an **849px file for a 12-inch print — 71 DPI, and unusable**. Nothing in either file made that visible, because neither one mentioned DPI at the point the number was chosen. Both now import from the same module, so the rule cannot drift again.

TARGET	FLOOR	MAX EDGE	UPLOAD BUDGET
300 DPI	150 DPI	8192 px	8.5 MB
STEP			
50 DPI			

Export steps the DPI down from 300 to 150 until the PNG fits the upload budget, then **reports the resolution it actually achieved** rather than the one it asked for — the edge clamp can land it lower. A photo-heavy design still produces a usable file instead of erroring at checkout.

Adding to cart then exports each side's print PNG, composites each side's preview onto the mockup, uploads all four, writes one immutable design row carrying the achieved DPI per side, and adds the cart line. If the lowest figure is under target the shopper is told at the moment it still matters:

```
toast.warning(  
  "Added – but your artwork exports at 143 DPI.  
  A larger source image will print sharper."  
)
```

Recording the achieved DPI per side is what lets production see what it is holding *without downloading and measuring it*.

The design row is immutable

Every "Add to cart" inserts a fresh row rather than updating one, so a design already sitting inside a placed order can never be retroactively changed by a later edit.

Security on the write

This was the only write in the customize flow with no checks at all — no schema, no rate limit, and it stored client-supplied URLs that the server later fetches and an admin later downloads. Its mobile twin validated every field. It now carries Zod validation on every field, a rate limit of 20 saves per 10 minutes, and every URL pinned to our own storage, which closes the SSRF. Guests can still design without an account — that is the point of the studio — so the control is *rate and shape*, not authorization.

Full parity with the web studio, including both doors: `start=library` opens straight into the shelf, so a link behaves the same wherever it is opened. Garment switching, carry and re-fit all exist, with the geometry deliberately identical to the web module.

Where it deliberately differs

- **The phone posts structured layer data, not images.** Rasterising on the server makes the print file resolution-independent and identical across iOS and Android — and it is required in practice, because on-device capture does not work reliably under React Native's New Architecture, which the app needs for Reanimated 4.
- **The renderer's fonts are the studio's fonts.** The server registers the exact TTFs the app bundles, under distinct family names so they cannot collide with a system font. Inter ships no italic in that package, so italic sans falls back to upright rather than letting the rasteriser synthesise a slant the phone never showed.
- **Print zones are server-side truth.** The route reads the config from the database and ignores any geometry in the request — taking it from the client would let someone print outside the area the garment allows.
- **Auth is optional but strict.** No token means guest, matching web. A token that *was* sent but does not verify is a 401, not a downgrade — silently treating it as a guest would attach someone's design to nobody.

Two rules the phone learned the hard way

PLACEMENT

The old rule scaled the image's longest edge to 70% of the zone's shortest edge and centred it — wrong twice over with real artwork. A wide design (a ridgeline is about 3.5:1) was sized against the 16in height while limited by the 12in width, so it landed far smaller than the space available; a tall design filled barely half the height it could; and everything sat in the vertical middle of a 16in box, which on a body is the navel rather than the chest. It is now contain-fit with a margin and a top bias — correct for every aspect ratio, and the upper third is where a chest print actually goes.

PRINT QUALITY

The studio captured the source image's width and height from the picker and used them *only for layout* — the numbers that decide whether a print is sharp were read, used, and thrown away. A shopper could take a 400px screenshot, stretch it across a 12in front, approve a preview that looked perfect at phone scale, and receive a garment printed at roughly **33 DPI**. Nothing said a word. Every image layer is now graded against the same thresholds while the shopper can still do something about it.

09 Data model

Four migrations carry the customisation half of this phase.

MIGRATION	WHAT IT ESTABLISHES
018	The foundation: <code>is_customizable</code> , <code>customization_config</code> , the immutable <code>custom_designs</code> row, and the cart and order links to it
092	<code>design_library</code> — the table, its constraints, its index, its RLS, and the public storage bucket
094	The edges joining studio to catalogue, plus the integrity that cannot be expressed as a column
095	The correction: the range is a flag, not a recipe

Modelling decisions worth keeping

- **No new entity for a printed garment.** A printed tee *is* a product and library artwork *is* a design. Two nullable foreign keys say the true thing directly; a join table would model a many-to-many that cannot occur, then need a uniqueness constraint to forbid the rows it just made possible.
- **`blank_ids` is an array, not a join table either.** The question is “may I show this artwork on that garment?”, asked once per studio session over a handful of blanks. An empty array means every blank — the common case, costing no rows — and a GIN index keeps the containment test cheap as the library grows.
- **`collection` is free text, not a foreign key.** Product collections are ranges of physical garments. A design collection is a different thing that happens to share the word.
- **The dropped column stayed dropped.** `library_design_id` only existed to support the recipe model. Nothing read it, and a column nothing reads is a question every future reader has to answer.

The database refuses the impossible

Rather than trusting every caller to remember:

- A product cannot be printed on itself.
- A parent blank without the range flag is a contradiction no screen can render, so a CHECK forbids it.
- A parent must be a *real* customizable blank with print zones — cross-row, so it needs a trigger, and it gets one. Deliberately not `SECURITY DEFINER`: it reads a table the caller already has open, so it needs no extra reach.

Because those refusals surface as constraint names, they are translated into sentences an admin can act on — “That blank is not customizable, or has no print zones set up. Pick a product with colourways configured on its Customization tab.”

Who can read and write

`design_library` carries one SELECT policy and **no write policies at all**. Reads are public because this is a catalogue we are advertising; writes go through server actions holding the service-role key, which bypasses RLS. Anything reaching that table with the public key can read the live rows and nothing more.

SECURITY WORK TOUCHING THIS SYSTEM	WHAT IT CLOSED
Catalogue lockdown	The product catalogue was writable by anybody — the table every customisation flag lives in
Guest-design hardening	The guest-design exposure, the SSRF in the design write, and three open writes
Profile RLS	Anyone with the public key could read and edit every profile
Definer functions	<code>SECURITY DEFINER</code> functions were callable with no session

The studio runs on its own palette, and that is correct

The first pass of the audit ranked the studio 5.5/10 on grep signals alone — no shared surface token, no shadow tokens, 28 ad-hoc radii — and concluded it was “styled as a utility”. Reading the code showed the opposite, and the score was revised to 7.5. The storefront's tokens are the wrong yardstick here.

It is a dark, **achromatic** system, and the reason is the whole argument:

The rest of the site is green, which is right for a brand about mountains and wrong for the one screen whose entire job is to judge colour.

A green cast sits next to the garment and the artwork and quietly lies about both: a warm print looks warmer against it, a grey marl looks green. **Selection carries no hue either** — it is signalled by luminance and an edge, a lifted surface and a bone hairline, which is how a tool says “this one” without spending a colour it may need to show you honestly.

One measured fix inside that set: the label tone was raised from 0.44 to 0.60 opacity, because the former measured **3.94:1** against the panel — under AA, and those are the 10px and 11px labels, the ones that most need to be readable. It now sits near 6:1.

What actually was wrong: the radii

22 × `rounded-sm`, 3 × `rounded-md`, 3 × `rounded-lg` — framework defaults, so a colour swatch, a tool button and the canvas frame all enclosed at roughly the same value and read as one class of object. Mapped onto the ladder by enclosure size: small controls, chips and swatch wells to the input rung; the canvas frame and picker panels to the panel rung. **28 → 0**. Scope, palette and layout unchanged.

The site-wide rule these sit inside

Cream is floor, not furniture.

A card is never the paper tone. A card is white with the card shadow and the card radius. The ground steps `paper → paper-warm → paper-deep` only when the *subject* changes; dark anchors break long scrolls; the dawn accent stays rare. This is a refinement of execution, not a re-pitch of the palette.

SIGNAL	BEFORE	AFTER
Files using the elevation ladder	3	18

SIGNAL	BEFORE	AFTER
Uses of the legal card fill	53	94
Off-ladder radii — app routes	~120	0
Off-ladder radii — components	~120	0
Bare <code>rounded</code> (the 4px default)	17	0
Undefined colour tokens	17	0
Invisible form fields	16	0
Shop filter tests	0	29
Design-system lint rules	0	4

Bugs found along the way

None of these were on the plan; they surfaced while doing the work.

- **Two colour names were never theme tokens** — used 17 times. In Tailwind v4 an undefined colour compiles to nothing, so the rental cancel button rendered *cream text on a transparent background*: an unreadable button on a destructive action.
- **Shop filters never reached the URL**. No router call at all, so a filtered shop could not be shared or bookmarked, back did not undo a filter, and opening a product and returning discarded the selection.
- **Sixteen form fields were invisible** — paper inputs on a paper page across checkout, rent, Trek Buddy and the plan console.
- **Order status was encoded in colour alone**, failing WCAG 1.4.1. The badge now carries dot-shape, label and colour, so any two of the three suffice.
- **The clay tone was carrying small text at 3.26:1**. Moved to the deeper rung the palette had already defined for exactly this, at 5.79:1.
- **Product cards with no photograph rendered as holes**. They now carry the topographic motif with the piece's initial in the display face, so a product awaiting its picture still looks like one.
- **Tailwind v4 scans `.md` and `.mjs`**. Twice during this work a class-shaped string — once in a design document, once in a lint rule message — generated invalid CSS and 500'd every route.

11 What shipped, when

105 commits across the window. These are the ones that built or moved this system.

DATE	CHANGE
16 Aug	Commerce engine — promotions, tax rules, returns, production and jobs. The rails a custom order runs on.
16 Aug	Storefront cached instead of rendered per visitor
17 Aug	Closed the guest-design exposure, the SSRF and three open writes
17 Aug	Order the size the customer picked, not the first one in the list
18 Aug	The customiser gets a surface ladder, an edge and readable type
18 Aug	Take the green out — a colour tool must not tint what it shows you
18 Aug	Definer-function and profile RLS lockdowns
20–21 Aug	Mobile: stop quoting a price the shop does not charge; two Android-only defects; the Android pass recorded
26 Aug	The 23 August client brief, end to end — the design library table, the admin screen, the studio picker, both doors named
26 Aug	The product catalogue was writable by anybody
27 Aug	Auth form split, nav rebuilt, sage lifted on dark
28–29 Aug	The custom range — the edges, then the correction to a flag; the blank switcher and carry; the DPI unification; mobile placement, quality and parity; both seed scripts

ADD ARTWORK TO THE LIBRARY

- Open **Design Library** in admin and use *Add a design*.
- Name it, set the collection — this is what groups it in the studio — and choose a PNG with a transparent background, at least 2000px on the long edge.
- It goes live at the end of the shelf. Change the sort number to move it.
- Leave *Offered on* as **Every garment** unless the artwork only suits one blank's zone.

TAKE ARTWORK DOWN

- Untick **Live** — it leaves the studio, the row and the file survive, and anything already ordered is untouched.
- Delete only when you want the file gone too.

PUT A PRINTED PRODUCT IN THE CUSTOM RANGE

- Open the product's **Customization** tab and leave *Customizable product* unticked — this is a print, not a blank.
- Tick **Part of the custom range**.
- Optionally set *Printed on* to the blank it came off. Leave it unset if you do not stock that blank; the page then offers the ones you do.

ADD A NEW BLANK

- Tick *Customizable product*.
- Add colourways with front and/or back print zones, and set the print width and height in inches — everything downstream is derived from those two numbers.

13 Completeness

Everything the brief asked for on customisation is built and connected.

- ✓ **Pre-set DEWDROPZ design library** — table, storage, access rules, admin CRUD, ordering, on/off, collections, per-garment scoping.
- ✓ **Customer uploads their own design** — unchanged, and hardened with schema validation, a rate limit and URL pinning.
- ✓ **Both doors named and routed** from the homepage, the index, every blank card, every custom-range product page and the phone app.
- ✓ **Back-end management** of the library on one screen, per-row saves, optimistic with rollback, orphan-safe in both directions.
- ✓ **Marking other products as customisation-related** — a plain tick with an optional parent blank, enforced in Postgres and explained in the admin.
- ✓ **The storefront card** with two honest states, sibling prints and a library-first entry point.
- ✓ **Full studio** — front and back, images, type, ink, layers, transforms, undo/redo, copy across sides, garment switching with geometric re-fit.
- ✓ **One print rule across three renderers**, 300 DPI target, 150 floor, achieved DPI recorded per side and surfaced to the shopper and to production.
- ✓ **Mobile parity**, with server-side rasterisation and matching fonts.
- ✓ **Its own accessible, achromatic design system**, zero off-ladder radii.
- ✓ **Security** — public reads, admin writes, server-side geometry, no client URLs reaching a server fetch.

No further work is required on customisation for this phase.

TWO THINGS NOTED FOR NEXT PHASE — NEITHER BLOCKING

1. The library needs dark-ink counterparts when a light colourway goes live. Today only Jet Black is orderable, so every generated mark is set in the brand's paper tone.

2. One dead reference to a dropped column. `library_design_id` still appears in the audited-fields list and in one branch of the write-error translator, after the migration dropped the column. It is inert — the audit logger reads a field the row no longer has, and the error branch can never match — but it is two lines of tidying whenever that file is next opened.